CSE321: Analysis and Design of Algorithms

Assignment 1 – Divide and Conquer (Fall 2026)

Student: Ali Amr Abdullah Ahmed | **ID:** 23010589

1 Problem 1: Median Selection

1.1 Implemented Methods

All three algorithms find the median by returning the element at zero-based index $k = \lfloor n/2 \rfloor$:

1. **Naive Sort Select:** Sorts the array using Java's native `Arrays.sort(arr)` and returns `arr[k]`. Time complexity: $O(n \log n)$.
2. **Randomized Select (CLRS 9.2):** Picks a random pivot using `ThreadLocalRandom`, partitions the subarray in-place around the pivot, and recurses only into the side containing index k. Expected time complexity: $O(n)$; worst-case: $O(n^2)$.
3. **Deterministic Select (CLRS 9.3 Median-of-Medians):** Divides the current subarray into groups of 5, sorts each group in-place using insertion sort, extracts each group's median into a `medians` array, and recursively selects the median of those medians as the pivot. Guarantees a balanced partition and worst-case $O(n)$ time on distinct elements.

1.2 Experimental Setup and Verification

- **Input Sizes:** Tested on randomly generated 32-bit integer arrays of sizes $n \in \{10^3, 10^4, 10^5, 5 \times 10^5, 10^6, 5 \times 10^6, 10^7\}$.
- **Warmup and Averaging:** The JVM is warmed up for 10 iterations on arrays of size 10^6 before timing begins. Each data point is the average of 50 benchmark runs.
- **Fairness and Correctness:** For each run, a single master random array is generated and cloned before calling each algorithm. Array allocation, cloning, and garbage collection hints occur outside the `System.nanoTime()` window. The return values of both selection methods are checked against the sorting baseline on every run, in addition to a separate unit test suite covering 500 random trials and edge cases.

1.3 Benchmark Results

Table 1: Average running time (in milliseconds) over 50 runs per input size.

n	Naive Sort (ms)	Randomized (ms)	Deterministic (ms)	Sort / Rand	Det / Rand
1,000	0.038	0.018	0.035	2.11×	1.94×
10,000	0.379	0.084	0.286	4.51×	3.40×
100,000	4.548	0.750	2.712	6.06×	3.62×
500,000	26.351	3.752	13.203	7.02×	3.52×
1,000,000	56.462	7.202	26.350	7.84×	3.66×
5,000,000	307.382	49.711	133.490	6.18×	2.69×
10,000,000	635.236	101.272	261.337	6.27×	2.58×

1.4 Performance Analysis

1. **Empirical Linear Scaling ($O(n)$):** Both selection algorithms scale linearly with n. Increasing n by a factor of 100× (from 10^5 to 10^7) increases the runtime of Randomized Select from 0.750 ms to 101.272 ms and Deterministic Select from 2.712 ms to 261.337 ms ($\approx 96.4\times$).

2. **Randomized vs. Deterministic Select:** Across all tested sizes, Randomized Select is 2.58× to 3.66× faster than Deterministic Select. While both are $O(n)$, Median-of-Medians incurs a larger constant factor due to sorting $\lceil n/5 \rceil$ groups of 5, allocating the `medians` array, recursively finding the pivot, and scanning for the pivot index before partitioning ($T(n) \leq T(n/5) + T(7n/10) + O(n)$ vs. expected $2n$ comparisons for Randomized Select).

3. **Selection vs. Naive Sorting:** On the standard JVM run in Table 1, Naive Sort (635.236 ms at $n = 10^7$) is 6.27× slower than Randomized Select and 2.43× slower than Deterministic Select because it fully orders both partitions at every level ($\Theta(n \log n)$). Note that on newer JVMs (e.g., JDK 21/25), `Arrays.sort(int[])` uses AVX2/AVX-512 SIMD hardware intrinsics (`libsimdsort`), which can drop the sorting time to ~ 70 ms at 10^7 and rival scalar Java selection loops despite doing more asymptotic operations.

2 Problem 2: Minimum-Perimeter Triangle

2.1 Divide-and-Conquer Approach

Given n distinct 2D points ($3 \leq n \leq 10^5$), we want to find three distinct points that minimize the perimeter $P(a, b, c) = d(a, b) + d(b, c) + d(c, a)$, including degenerate collinear triangles.

1. **Pre-sorting:** Before recursion, create two copies of the input array: X sorted lexicographically by (x, y), and Y sorted lexicographically by (y, x).

2. **Base Case ($n \leq 5$):** When the subarray has at most 5 points, compute the minimum perimeter directly by checking all $\binom{n}{3} \leq 10$ triples in $O(1)$ time. Because $n \geq 6$ in recursive calls, both halves always contain at least 3 points.

3. **Divide:** Split X at index mid $= \lfloor n/2 \rfloor$ into X_L and X_R around midPoint $= X[\text{mid} - 1]$. Partition Y in a single $O(n)$ pass into Y_L and Y_R using the exact same lexicographic condition ($p.x <$ midPoint.x or ($p.x ==$ midPoint.x and $p.y \leq$ midPoint.y)), preserving y-sorted order even when multiple points share the same x-coordinate.

4. **Conquer:** Recursively solve both halves to obtain $\delta_1 = \text{minPerimeter}(X_L, Y_L)$ and $\delta_2 = \text{minPerimeter}(X_R, Y_R)$, and set $\delta = \min(\delta_1, \delta_2)$.

5. **Combine (Strip Search):** If a triangle crossing the dividing line has perimeter $P < \delta$, then by the triangle inequality every side length is strictly less than $P/2 < \delta/2$ (since $2a \leq a+b+c = P < \delta \implies a < \delta/2$). We filter Y in $O(n)$ time into Y_{strip}, keeping only points with $|p.x - \text{midPoint}.x| < \delta/2$. Then, iterating through Y_{strip} in ascending y-order, for each point $Y_{\text{strip}}[i]$ we test subsequent points $Y_{\text{strip}}[j]$ and $Y_{\text{strip}}[k]$ ($i < j < k$), breaking out of the inner loops immediately when y-distance from $Y_{\text{strip}}[i]$ reaches or exceeds $\delta/2$.

2.2 Time Complexity Analysis

- **Initial Pre-sorting:** Sorting X and Y takes $O(n \log n)$ time.
- **Bounding the Strip Search ($O(n)$):** For any point $Y_{\text{strip}}[i]$, any candidate points $Y_{\text{strip}}[j]$ and $Y_{\text{strip}}[k]$ that form a triangle with perimeter $< \delta$ must lie inside a $\delta \times (\delta/2)$ bounding box above $Y_{\text{strip}}[i]$ (comprising a $(\delta/2) \times (\delta/2)$ square on the left of the dividing line and a $(\delta/2) \times (\delta/2)$ square on the right).

Partition each $(\delta/2) \times (\delta/2)$ square into a 2×2 grid of four sub-squares of side length $s = \delta/4$ (8 sub-squares in total across both sides). The maximum perimeter of any three points inside a single $(\delta/4) \times (\delta/4)$ sub-square is bounded by a right triangle at its corners:

$$P_{\max} = s + s + s\sqrt{2} = \frac{2+\sqrt{2}}{4}\delta \approx 0.8535\delta < \delta.$$

Since every sub-square lies entirely within the left half or entirely within the right half, and the minimum perimeter within either half is δ, at most 2 points can reside in any $(\delta/4) \times (\delta/4)$ sub-square. With 8 sub-squares total, the $\delta \times (\delta/2)$ window contains at most $8 \times 2 = 16$ points. Thus, for each index i, the inner loops over j and k execute at most $\binom{15}{2} = 105 = O(1)$ iterations before the y-coordinate difference exceeds $\delta/2$ and triggers `break`.

- **Overall Recurrence:** Splitting X and Y, building Y_{strip}, and scanning the strip all take $O(n)$ time:

$$T(n) = 2T(n/2) + O(n) \implies \boxed{T(n) = O(n \log n)}.$$

Appendix A: Problem 1 Source Code

SelectionAlgorithm.java

```java
package problem1;

public interface SelectionAlgorithm {
    int select(int[] arr, int k);
    String getName();
}
```

NaiveSortSelect.java

```java
package problem1;

import java.util.Arrays;

public class NaiveSortSelect implements SelectionAlgorithm {

    @Override
    public int select(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k];
    }

    @Override
    public String getName() {
        return "Naive Sort Select";
    }
}
```

RandomizedSelect.java

```java
package problem1;

import java.util.concurrent.ThreadLocalRandom;

public class RandomizedSelect implements SelectionAlgorithm {

    @Override
    public int select(int[] arr, int k) {
        return randomizedSelect(arr, 0, arr.length - 1, k);
    }

    private int randomizedSelect(int[] arr, int left, int right, int k) {
        if (left == right) {
            return arr[left];
        }
        int i = randomizedPartition(arr, left, right);
        if (i == k) return arr[i];
        else if (i > k) {
            return randomizedSelect(arr, left, i - 1, k);
        } else {
            return randomizedSelect(arr, i + 1, right, k);
        }
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
```

```
        arr[j] = temp;
    }

    private int randomizedPartition(int[] arr, int left, int right) {
        int r = ThreadLocalRandom.current().nextInt(left, right + 1);
        swap(arr, left, r);
        int j = left + 1;
        for (int i = left + 1; i <= right; i++) {
            if (arr[i] < arr[left]) {
                swap(arr, i, j);
                j++;
            }
        }
        j--;
        swap(arr, left, j);
        return j;
    }

    @Override
    public String getName() {
        return "Randomized D&C Select";
    }
}
```

DeterministicSelect.java

```
package problem1;

public class DeterministicSelect implements SelectionAlgorithm {

    @Override
    public int select(int[] arr, int k) {
        return deterministicSelect(arr, 0, arr.length - 1, k);
    }

    private int deterministicSelect(int[] arr, int left, int right, int k) {
        if (left == right) {
            return arr[left];
        }
        int pivot = medianOfMedians(arr, left, right);
        int i = partition(arr, left, right, pivot);
        if (i == k) return arr[i];
        else if (i > k) {
            return deterministicSelect(arr, left, i - 1, k);
        } else {
            return deterministicSelect(arr, i + 1, right, k);
        }
    }

    private int medianOfMedians(int[] arr, int left, int right) {
        int gps = (right - left) / 5 + 1;
        int[] medians = new int[gps];
        for (int i = 0; i < gps; i++) {
            int groupLeft = left + i * 5;
            int groupRight = Math.min(groupLeft + 4, right);
            insertionSort(arr, groupLeft, groupRight);
            int mid = groupLeft + (groupRight - groupLeft) / 2;
            medians[i] = arr[mid];
        }
        return deterministicSelect(medians, 0, gps - 1, gps / 2);
    }

    private void insertionSort(int[] arr, int left, int right) {
```

```
        for (int i = left + 1; i <= right; i++) {
            int j = i - 1;
            while (j >= left && arr[j] > arr[j + 1]) {
                swap(arr, j, j + 1);
                j--;
            }
        }
    }

    private int partition(int[] arr, int left, int right, int pivot) {
        for (int i = left; i <= right; i++) {
            if (arr[i] == pivot) {
                swap(arr, left, i);
                break;
            }
        }
        int j = left + 1;
        for (int i = left + 1; i <= right; i++) {
            if (arr[i] < arr[left]) {
                swap(arr, i, j);
                j++;
            }
        }
        j--;
        swap(arr, j, left);
        return j;
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    @Override
    public String getName() {
        return "Deterministic Select (median of medians)";
    }
}
```

Main.java

```
package problem1;

import java.util.List;
import java.util.Locale;
import java.util.Random;

public class Main {

    private static final int WARMUP_RUNS = 10;
    private static final int BENCHMARK_RUNS = 50;
    private static final int[] INPUT_SIZES = {
        1_000, 10_000, 100_000, 500_000, 1_000_000, 5_000_000, 10_000_000
    };

    public static void main(String[] args) {
        List<SelectionAlgorithm> algorithms = List.of(
            new NaiveSortSelect(),
            new RandomizedSelect(),
            new DeterministicSelect()
        );

```

```
22          warmupJvm(algorithms);
23          runBenchmarks(algorithms);
24      }
25
26      private static void warmupJvm(List<SelectionAlgorithm> algorithms) {
27          System.out.println("Warming up JVM...");
28          Random rand = new Random(42);
29          int warmupSize = 1_000_000;
30          int k = warmupSize / 2;
31
32          for (int r = 0; r < WARMUP_RUNS; r++) {
33              int[] master = generateRandomArray(warmupSize, rand);
34              for (SelectionAlgorithm algo : algorithms) {
35                  int[] copy = master.clone();
36                  algo.select(copy, k);
37              }
38          }
39          System.out.println("Warmup complete.\n");
40      }
41
42      private static void runBenchmarks(List<SelectionAlgorithm> algorithms) {
43          Random rand = new Random(100);
44
45          System.out.printf("%-12s | %-28s | %-28s | %-28s%n",
46                  "N",
47                  algorithms.get(0).getName(),
48                  algorithms.get(1).getName(),
49                  algorithms.get(2).getName());
50          System.out.println("-".repeat(105));
51
52          for (int n : INPUT_SIZES) {
53              int k = n / 2;
54              long[] totalTimeNanos = new long[algorithms.size()];
55
56              for (int run = 0; run < BENCHMARK_RUNS; run++) {
57                  int[] master = generateRandomArray(n, rand);
58                  Integer baselineResult = null;
59
60                  for (int i = 0; i < algorithms.size(); i++) {
61                      int[] copy = master.clone();
62                      System.gc();
63
64                      long start = System.nanoTime();
65                      int result = algorithms.get(i).select(copy, k);
66                      long elapsed = System.nanoTime() - start;
67                      totalTimeNanos[i] += elapsed;
68
69                      if (baselineResult == null) {
70                          baselineResult = result;
71                      } else if (result != baselineResult) {
72                          throw new IllegalStateException("Mismatch at N=" + n + " from " +
    algorithms.get(i).getName());
73                      }
74                  }
75              }
76
77              double avg0Ms = (totalTimeNanos[0] / (double) BENCHMARK_RUNS) / 1_000_000.0;
78              double avg1Ms = (totalTimeNanos[1] / (double) BENCHMARK_RUNS) / 1_000_000.0;
79              double avg2Ms = (totalTimeNanos[2] / (double) BENCHMARK_RUNS) / 1_000_000.0;
80
81              System.out.printf(Locale.US, "%-12d | %25.3f ms | %25.3f ms | %25.3f ms%n",
82                      n, avg0Ms, avg1Ms, avg2Ms);
83          }
84      }
85
86      private static int[] generateRandomArray(int size, Random rand) {
```

```
87          int[] arr = new int[size];
88          for (int i = 0; i < size; i++) {
89              arr[i] = rand.nextInt();
90          }
91          return arr;
92      }
93  }
```

Appendix B: Problem 2 Source Code

Point.java

```java
package problem2;

public class Point {
    public final long x;
    public final long y;

    public Point(long x, long y) {
        this.x = x;
        this.y = y;
    }

    public double distanceTo(Point other) {
        long dx = this.x - other.x;
        long dy = this.y - other.y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}
```

MinPerimeterSolver.java

```java
package problem2;

import java.util.Arrays;

public class MinPerimeterSolver {

    public double solve(Point[] points) {
        Point[] X = points.clone();
        Arrays.sort(X, (a, b) -> {
            if (a.x != b.x) return Long.compare(a.x, b.x);
            return Long.compare(a.y, b.y);
        });
        Point[] Y = points.clone();
        Arrays.sort(Y, (a, b) -> {
            if (a.y != b.y) return Long.compare(a.y, b.y);
            return Long.compare(a.x, b.x);
        });
        return minPerimeter(X, Y);
    }

    private double minPerimeter(Point[] X, Point[] Y) {
        int n = X.length;
        if (n <= 5) {
            return baseCase(X);
        }
        int mid = n / 2;
        Point midPoint = X[mid - 1];
        Point[] XL = Arrays.copyOfRange(X, 0, mid);
        Point[] XR = Arrays.copyOfRange(X, mid, n);
        Point[] YL = new Point[mid];
        Point[] YR = new Point[n - mid];
        int li = 0, ri = 0;
        for (Point p : Y) {
            if (p.x < midPoint.x || (p.x == midPoint.x && p.y <= midPoint.y)) {
                YL[li++] = p;
            } else {
                YR[ri++] = p;
            }
```

```
        }
        double delta1 = minPerimeter(XL, YL);
        double delta2 = minPerimeter(XR, YR);
        double delta = Math.min(delta1, delta2);

        Point[] Y_strip = new Point[n];
        int size = 0;
        for (Point p : Y) {
            if (Math.abs(p.x - midPoint.x) < delta / 2.0) {
                Y_strip[size++] = p;
            }
        }
        for (int i = 0; i < size; i++) {
            for (int j = i + 1; j < size; j++) {
                if (Y_strip[j].y - Y_strip[i].y >= delta / 2.0) {
                    break;
                }
                for (int k = j + 1; k < size; k++) {
                    if (Y_strip[k].y - Y_strip[i].y >= delta / 2.0) {
                        break;
                    }
                    double p = perimeter(Y_strip[i], Y_strip[j], Y_strip[k]);
                    if (p < delta) {
                        delta = p;
                    }
                }
            }
        }
        return delta;
    }

    private double baseCase(Point[] pts) {
        double minP = Double.POSITIVE_INFINITY;
        int n = pts.length;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                for (int k = j + 1; k < n; k++) {
                    double p = perimeter(pts[i], pts[j], pts[k]);
                    if (p < minP) {
                        minP = p;
                    }
                }
            }
        }
        return minP;
    }

    private double perimeter(Point a, Point b, Point c) {
        return a.distanceTo(b) + b.distanceTo(c) + c.distanceTo(a);
    }
}
```